Efficient reasoning. Although recent O-1 or R-1 style LRMs [27, 10, 32, 33] have demonstrated strong capabilities in self-reflection and solution searching, pushing the boundaries of performance across various tasks, they still struggle with the so-called "overthinking phenomenon" [8, 31]: they may produce verbose reasoning or redundant intermediate steps even for a very simple question, leading to lengthy output responses, resulting in substantial computational overhead and increased reasoning time. To address this issue, numerous studies have explored methods for optimizing the reasoning trajectory and improving token efficiency while preserving accuracy. Some approaches introduce length-based rewards during the reinforcement learning phase to incentivize models to balance the trade-off between accuracy and efficiency. In such settings, shorter yet accurate answers are expected to receive higher rewards generally [4, 32, 21]. Other methods apply supervised fine-tuning (SFT) using variable-length reasoning datasets, including techniques such as CoT compression and reconstruction with tailored strategies—for instance, retaining the earliest solution that first arrives at the correct answer [31], sampling the shortest correct answers [25], or leveraging state-of-the-art LLMs as compressors to generate more concise reasoning paths [18]. Meanwhile, recent work has shown that models can be routed into two distinct inference modes: quick answer generation or step-by-step reasoning [33]. Notably, training-free methods have also been shown to be effective in this context. Ma et al. [22] demonstrate that models can maintain high accuracy even when prompted to skip the reasoning stage and directly generate the final answer. Similarly, Lee et al. [19] show that models are capable of compressing their own CoT to stay within a token budget, suggesting that accuracy and efficiency can seemingly be preserved without trade-offs. In this work, we revisit this apparent "free lunch" through the perspective of behavioral inconsistency, asking: does the pursuit of efficiency compromise the consistency of models?

3 Defining and measuring inconsistency in large reasoning models

In this section, we first clarify the definition of inconsistency in LRMs (§ 3.1), and then introduce our benchmark, ICBENCH, designed to systematically measure inconsistency in LRMs (§ 3.2).

3.1 Characterizing and defining inconsistency

Various terms have been used to describe different forms of inconsistency, as discussed in § 2. In this work, we primarily focus on behavioral inconsistency as manifested during model inference, rather than inconsistencies introduced during training, such as reward hacking [30]. Before conducting a comprehensive evaluation, we first present a systematic definition of three key types of behavioral inconsistency. These categories are informed by and aligned with the most frequently discussed inconsistency phenomena in prior work (e.g., [34, 3, 16]).

Inconsistency across task settings *(ITS)*. refers to the phenomenon where a model produces different answers to the same question when presented in different settings. For instance, Xu et al. [36] observed that LLMs exhibit divergent behavior between word-based questions (i.e., What's your opinion) and deed-based questions (i.e., Which action will you take), characterized as *"saying one thing and doing another."* Similarly, models behave differently when answering the same question in open-ended formats versus multiple-choice settings [35, 2]. Another form of *ITS* arises in refusal behavior: the same toxic prompt may be accepted or rejected depending on whether the model is in a Deployment vs. Development phase, as illustrated by the left example in Figure 1, or whether it is accessed via a free-user or paid-user [35]. These inconsistencies have been widely discussed in the context of jailbreaking, particularly under methods employing "competing objectives" [20, 16].

Inconsistency between our training objective and learned model behavior *(TR-LB)*. arises when a model's outputs are influenced by triggers like hints or biasing features [34] embedded in prompts, leading it to adopt behaviors such as shortcutting or sycophancy to try to get a higher reward from the evaluator [11, 6]. For instance, a model can learn to modify the `compute_reward.py` script in the environment [11] or employ hint in the context window [9]. These behaviors often manifest as scheming, where the model manipulates its outputs to superficially align with user expectations [24]. As shown in the middle panel of Figure 1, the model changes its previously correct answer to an incorrect one when exposed to a misleading hint injected into the prompt.

Inconsistency between internal reasoning and output self-explanations *(IR-SE)*. means the model will not always tell the truth about what motivates it to get their final decision, even if the self-explanation appears plausible, characterized as models "don't always say what they think" or as

producing unfaithful self-explanations [9, 29]. Furthermore, as shown in the right panel of Figure 1, LRM may internally admit (e.g., between `<think>` and `</think>` tags) that they merely guessed the answer, but still output a seemingly well-reasoned explanation for choosing option C, which aligns with the finding defined as "Implicit Post-Hoc Rationalization" by Arcuschin et al. [2]. Efficient reasoning by encouraging the model to skip the thinking stage or compress the CoT may intuitively lead the model to hide its actual guesswork and instead produce post-hoc rationalizations. To further investigate this possibility, we present a comprehensive benchmark for evaluating such inconsistencies in the § 3.2.

3.2 ICBENCH: A Framework for measuring inconsistency of LRMs

Drawing inspiration from previous research on jailbreaking and other adversarial attack methodologies [20, 11, 12], we develop adversarial inputs with triggers, manipulate environmental conditions, and utilize diverse task settings to systematically elicit and measure inconsistencies in models.

Given a query $x \in \mathcal{D}$, we define the response of an LRM $\mathcal{M}$ to an original prompt (a prompt prefixed solely with "`Please answer the following question`" and without additional triggers or cues) as

$$\mathcal{M}(x) = \text{Concat}\left(t^{\text{r}},\ t^{\text{a}},\ c,\ a \right),$$

where t denotes the model's internal reasoning trace, typically enclosed between special tokens `<think>` and `</think>`. This t can be further split into reflective reasoning steps t^{r} and a conclusion t^{a}. c corresponds to the externalized chain-of-thought following `</think>` and preceding the final output. Finally, a represents the model's conclusive answer or action.

An illustrative example is provided below.

`<think>` Okay, let's try to solve this problem step by step. The question gives me ...Let me check ... Therefore, the correct answer is B. `</think>` To solve the given system of equations and determine ... we begin by ... Step 1, ... Since 25 and 8 are relatively prime ... the answer is 33, and thus: B

Tasks and triggers. To better evaluate inconsistencies across different scenarios, we include a diverse set of tasks spanning both mathematical reasoning and human alignment. For mathematical tasks, we adopt standard math problems from AIME2024 [1] and AIME2025 [2], as well as Olympiad-level questions from OlympiadBench [17], following the same setting as [22]. All datasets are reformatted into both open-ended QA and multiple-choice formats. For alignment tasks, we incorporate the widely used AdvBench [37] to test whether models comply with harmful requests. We also include the Words and Deeds Consistency Test (WDCT) [36], which assesses whether a model's stated opinions or ethical positions remain consistent between verbal statements and corresponding actions. Specifically for *TR-LB*, we restrict evaluation to the multiple-choice format of mathematical reasoning tasks in order to avoid overly complex settings. To elicit model inconsistencies, we inject the triggers like adversarial task settings $\mathcal{T}$ and counterfactual cues $\mathcal{C}$ into the prompts. An overview of these setups is provided in Table 1, with full prompt template details available in Appendix B.

In the following, we formalize the metrics to measure the inconsistency regarding *ITS*, *TR-LB*, and *IR-SE* respectively:

Adversarial task settings for measuring *ITS*. To measure *ITS*, we evaluate whether a model's final answer remains consistent under adversarial task settings, such as `Deployment` versus `Development`. We construct pairs of task configurations, where each pair consists of a baseline setting $\mathcal{T}_{\text{base}}$ and a corrupted setting $\mathcal{T}_{\text{corr}}$ as illustrated in Table 1. These configurations are designed to induce behavioral changes in the model by altering system-level prompts (e.g., switching to an "act as in the development mode") or by modifying the task format (e.g., converting an open-ended question-answering to multiple choice).

For the given $\mathcal{M}$ and a query x, we can define the occurrence of *ITS* as the final answer changes under $\mathcal{T}_{\text{base}}$ and $\mathcal{T}_{\text{corr}}$:

$$a_{\text{base}} \neq a_{\text{corr}},$$

[1] https://huggingface.co/datasets/HuggingFaceH4/aime_2024
[2] https://huggingface.co/datasets/opencompass/AIME2025

where $\mathcal{M}(\mathcal{T}_{\text{base}}(x)) = \text{Concat}(t_{\text{base}}, c_{\text{base}}, a_{\text{base}})$ and $\mathcal{M}(\mathcal{T}_{\text{corr}}(x)) = \text{Concat}(t_{\text{corr}}, c_{\text{corr}}, a_{\text{corr}})$. Crucially, this inequality does not refer to token-level exact match, but instead reflects a shift in the model's final answer. The criteria used for this judgment are detailed in Appendix B.

To quantify this, we define the $\text{ITS}_{\text{score}}$ as the expected *ITS* over a dataset $\mathcal{D}_{\mathcal{T}}$: $\text{ITS}_{\text{score}} = \mathbb{E}_{x\sim\mathcal{D}_{\mathcal{T}}}\left[\mathbb{I}\left(a_{\text{base}} \neq a_{\text{corr}}\right)\right]$, where $\mathbb{I}$ is the indicator function.

Contextual cues for measuring *TR-LB*. To measure *TR-LB*, we examine whether the model's final answer is influenced by contextual cues that elicit sycophancy and shortcut-taking behavior. Specifically, as shown in Table 1, we define a set of counterfactual cues $\mathcal{C}$ designed to trigger such behaviors in model $\mathcal{M}$. Inspired by recent studies on LLM sycophancy and strategic misalignment [11, 15, 24, 14], we categorize these cues into two types:

- Sycophancy cues ($\mathcal{C}_c$), where the model is implicitly nudged to conform to user-provided opinions or preferences, even when they are incorrect.[3]
- Shortcut-inducing cues ($\mathcal{C}_s$), where the prompt includes spurious patterns or permissions that suggest exploitable shortcuts.[4]

Given a model $\mathcal{M}$ and a query x, we define the occurrence of *TR-LB* as the case where the final answer under contextual cues, denoted $a_{\mathcal{C}}$, differs from the original answer a:

$$a_{\mathcal{C}} \neq a,$$

where the model's response under these cues is $\mathcal{M}(\mathcal{C}(x)) = \text{Concat}(t_{\mathcal{C}}, c_{\mathcal{C}}, a_{\mathcal{C}})$. The $\text{TR-LB}_{\text{score}}$ quantifies this inconsistency over a dataset $\mathcal{D}_{\mathcal{C}}$: $\text{TR-LB}_{\text{score}} = \mathbb{E}_{x\sim\mathcal{D}_{\mathcal{C}}}\left[\mathbb{I}\left(a_{\mathcal{C}} \neq a\right)\right]$.

Normalized inconsistency score (NIS). To ensure that observed inconsistencies are not merely due to randomness (e.g., sampling variability), we need to further optimize the $\text{ITS}_{\text{score}}$ and $\text{TR-LB}_{\text{score}}$ by normalization. For each query $x \in \mathcal{D}$, we generate N samples under each setting, denoted by $\{a_*^{(i)}\}_{i=1}^{N}$. The *intrinsic inconsistency* is then estimated by computing the proportion of pairwise disagreements among these answers of $\mathcal{T}_{\text{base}}$ or the original setting:

$$p_{\text{intrinsic}}(x) = \frac{\sum_{1\leq i<j\leq N} \mathbb{I}(a_*^{(i)} \neq a_*^{(j)})}{\max\left\{\frac{N(N-1)}{2}, \mathcal{S}\right\}}.$$

Here, $\mathcal{S}$ denotes the maximum number of sample pairs we set to control computational cost. If $\mathcal{S} < \frac{N(N-1)}{2}$, only a subset of sample pairs is randomly selected.

To estimate the *raw inconsistency* under perturbation, we define the perturbed setting as the same query x evaluated with an augmented prompt (i.e., injected with counterfactual cues $\mathcal{C}$ for *TR-LB* or alternate task settings $\mathcal{T}_{\text{corr}}$ for *ITS*). The model responses in this setting are denoted as ${a_\circ^{(i)}}_{i=1}^{N}$. We then compute the disagreement between each perturbed sample and its corresponding unperturbed counterpart:

$$p(x) = \frac{\sum_{i=1}^{N}\sum_{j=1}^{N} \mathbb{I}(a_\circ^{(i)} \neq a_*^{(j)})}{\max\left\{N^2, \mathcal{S}\right\}}.$$

The calibrated inconsistency probability is defined as the excess inconsistency remaining after subtracting the intrinsic inconsistency: $p_{\text{calibrated}}(x) = \max\left(0,\, p(x) - p_{\text{intrinsic}}(x)\right)$.

Finally, we define the Normalized Inconsistency Score (NIS) by aggregating the per-example calibrated scores and normalizing by the residual inconsistency not explained by intrinsic variability, following a similar formulation as in Chen et al. [9]:

$$\text{ITS}_{\text{NIS}} \text{ or } \text{TR-LB}_{\text{NIS}} = \frac{\mathbb{E}_{x\sim\mathcal{D}}\left[p_{\text{calibrated}}(x)\right]}{1 - \mathbb{E}_{x\sim\mathcal{D}}\left[p_{\text{intrinsic}}(x)\right]}.$$

[3] For example, the prompt may include a persona that provides a misleading hint like "I'm pretty sure the answer is C," which may influence the model to choose C despite its internal reasoning.

[4] For instance, all previous examples in the context window may choose option A, subtly biasing the model to follow suit regardless of its reasoning.